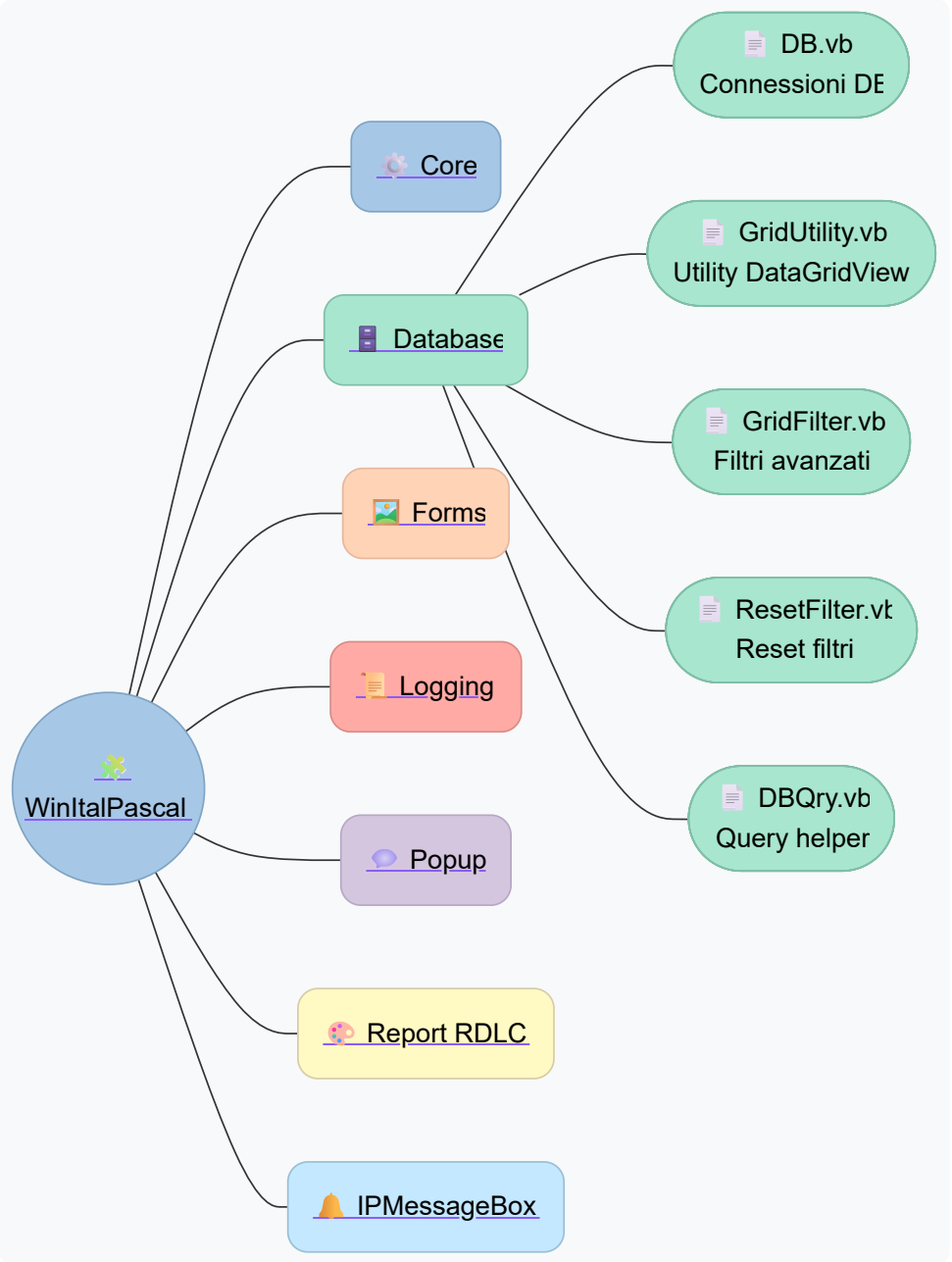
Ital Pascal


WinItalPascal
Core
Database
Forms
Logging
Popup
Report RDLC
IPMessageBox
DB.vb
Connessioni DE
GridUtility.vb
Utility DataGridView
GridFilter.vb
Filtri avanzati
ResetFilter.vb
Reset filtri
DBQry.vb
Query helper



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Spiegazione del modulo `ModCaricaDGV`

1. Scopo generale del codice

Il modulo **ModCaricaDGV** fornisce tre procedure (`CaricaDGV`, `CaricaDGV2`, `CaricaDGV3`) che consentono di caricare i risultati di una query SQL in un controllo `DataGridView` di un'applicazione Windows Forms.
In pratica, è un helper per eseguire query su un database SQL Server e visualizzare i dati in una griglia, con supporto per:

- query senza parametri (`CaricaDGV`);
- query con un numero variabile di parametri (`CaricaDGV2`);
- query con due gruppi di parametri (ad esempio per combinare `AND` o `OR`) (`CaricaDGV3`).

2. Componenti chiave e funzioni

Funzione	Descrizione	Parametri	Note
`CaricaDGV`	Esegue una query SQL senza parametri e imposta il `DataSource` del `DataGridView`.	`ByRef DGV As DataGridView`, `ByVal SQL As String`	Utilizza `SqlDataAdapter` per riempire un `DataSet`.
`CaricaDGV2`	Esegue una query con parametri.	`ByRef DGV As DataGridView`, `ByVal SQL As String`, `ByVal Parametri() As String`, `ByVal Valori() As String`	Aggiunge i parametri al comando con `AddWithValue`.
`CaricaDGV3`	Simile a `CaricaDGV2` ma permette due gruppi	`ByRef DGV As DataGridView`, `ByVal SQL As String`, `ByVal`	Gestisce due cicli `For` per

Funzione	Descrizione	Parametri	Note
	di parametri (utile per query più complesse).	`Parametri() As String`, `ByVal Valori() As String`, `ByVal Parametri2() As String`, `ByVal Valori2() As String`	aggiungere i parametri.

Struttura comune a tutte le funzioni

1. **Connessione al database**

```
Dim connString As String = ConfigurationManager.ConnectionStrings("MiaConnessione").ConnectionString
Dim conn As New SqlConnection(connString)
```

   Il nome della stringa di connessione è definito nel file di configurazione (`app.config` o `web.config`).
2. **Creazione di `SqlDataAdapter`**

```
Dim da As New SqlDataAdapter(SQL, conn)
```

   L'adapter esegue la query e riempie un `DataSet`.
3. **Gestione dei parametri** (solo `CaricaDGV2` e `CaricaDGV3`)

```
For i As Integer = 0 To Parametri.Length - 1
    da.SelectCommand.Parameters.AddWithValue(Parametri(i), Valori(i))
Next
```

4. **Riempimento del `DataSet` e assegnazione al `DataGridView`**

```
da.Fill(ds, "tabella")
DGV.DataSource = ds.Tables("tabella")
```

5. **Gestione degli errori**

```
Catch ex As Exception
    MsgBox(ex.Message)
End Try
```

3. Flusso di esecuzione

1. **Chiamata della procedura**
   L'applicazione (ad esempio un pulsante di ricerca) chiama una delle funzioni, passando:
   - il controllo `DataGridView` dove visualizzare i risultati;
   - la stringa SQL;
   - (opzionalmente) array di nomi e valori dei parametri.

2. **Connessione al database**
   La procedura apre la connessione con `conn.Open()`.
3. **Preparazione del comando**
   Viene creato un `SqlDataAdapter` con la query. Se ci sono parametri, vengono aggiunti al comando.
4. **Esecuzione della query**
   `da.Fill(ds, "tabella")` esegue la query e riempie il `DataSet` con una tabella chiamata `"tabella"`.
5. **Aggiornamento della griglia**
   Il `DataSource` del `DataGridView` viene impostato sulla tabella appena riempita, rendendo visibili i risultati.
6. **Chiusura della connessione**
   `conn.Close()` viene chiamato al termine, anche in caso di eccezione grazie al blocco `Try/Catch`.

---

4. Esempio di utilizzo

```
' Esempio di ricerca con un parametro
Private Sub RjBtnCerca_Click(sender As Object, e As EventArgs) Handles RjBtnCerca.Click
    Dim sql As String = "SELECT * FROM clienti WHERE cliente LIKE '%' + @nome + '%'"
    Dim parametri() As String = {"@nome"}
    Dim valori() As String = {TxtNome.Text}
    CaricaDGV2(CDataGrid, sql, parametri, valori)
End Sub
```

In questo caso la procedura `CaricaDGV2` aggiunge il parametro `@nome` e visualizza i risultati nella griglia `CDataGrid`.

> ✏️ 🔹 **Conclusione**
>
> Il modulo `ModCaricaDGV` è un helper semplice ma potente per caricare dati da SQL Server in un `DataGridView`. Con le tre versioni di `CaricaDGV` è possibile gestire query di diversa complessità, mantenendo il codice pulito e riutilizzabile.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms



🔹 Scopo generale del codice

Il frammento è una **funzione di utilità** che restituisce un oggetto `SqlConnection` pronto per essere usato in un'applicazione .NET.
La funzione è dichiarata `Shared`, quindi può essere chiamata senza creare un'istanza della classe che la contiene.
Il suo obiettivo è:

1. **Recuperare** la stringa di connessione dal file di configurazione (`App.config` o `Web.config`).
2. **Verificare** che la stringa esista.
3. **Creare** e **ritornare** un nuovo oggetto `SqlConnection` con la stringa ottenuta.

Componenti chiave

Elemento	Descrizione
`Public Shared Function GetConnection(Optional name As String = "MiaConnessione") As SqlConnection` Il valore di default è `"MiaConnessione"`.	Metodo pubblico e statico. Accetta un parametro opzionale `name` che indica il nome della stringa di connessione da cercare.
`ConfigurationManager.ConnectionStrings(name)` Richiede `Imports System.Configuration`.	Accesso alla collezione di stringhe di connessione del file di configurazione.
`If setting Is Nothing Then` L'eccezione è di tipo `Exception` con un messaggio in italiano.	Controllo di esistenza. Se la stringa non è

Elemento	Descrizione
	presente, viene lanciata un'eccezione.
`New SqlConnection(setting.ConnectionString)` Richiede `Imports System.Data.SqlClient`	Costruzione dell'oggetto `SqlConnection` con la stringa recuperata.

Flusso di esecuzione

1. **Chiamata**: `GetConnection()` viene invocato, con o senza parametro `name`.
2. **Recupero della configurazione**:

```
Dim setting = ConfigurationManager.ConnectionStrings(name)
```

   - Se `name` è `"MiaConnessione"`, cerca quella chiave nella sezione `<connectionStrings>` del file di configurazione.
3. **Verifica**:

```
If setting Is Nothing Then
    Throw New Exception($"Connection string '{name}' non trovata in App.config")
End If
```

   - Se la chiave non esiste, viene sollevata un'eccezione con un messaggio chiaro.
4. **Creazione della connessione**:

```
Return New SqlConnection(setting.ConnectionString)
```

   - Viene istanziato un nuovo `SqlConnection` con la stringa di connessione ottenuta.
5. **Restituzione**: L'oggetto `SqlConnection` è restituito al chiamante, pronto per essere aperto (`Open`) e utilizzato.

> ### 🔹 Conclusione
>
> Il codice fornisce un modo semplice e centralizzato per ottenere una connessione SQL a partire da una stringa di connessione definita nel file di configurazione. È utile in progetti dove si desidera evitare di hard-codificare le stringhe di connessione in più punti del codice..



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

🔹 Analisi della classe `DB` (VB.NET)

1. Scopo generale del codice

La classe `DB` è un *utility* statica (non instanziabile) che fornisce un set di metodi di accesso ai dati per un'applicazione .NET che utilizza **SQL Server**.
Il suo obiettivo è:

- **Centralizzare** la gestione delle connessioni e delle query.
- Offrire **metodi di alto livello** (scalar, non-query, reader, DataTable, DataSet) per eseguire SQL in modo semplice e sicuro.
- Fornire **funzioni di supporto** per la creazione dinamica di parametri e per le query "LIKE" con più parametri.

In pratica, è un "wrapper" intorno a `System.Data.SqlClient` che riduce il boilerplate di apertura/chiusura connessioni e di gestione dei parametri.

---

2. ℹ️ Componenti chiave e funzioni

Sezione	Funzione	Descrizione
Connessione	`GetConnection(Optional name As String = "MiaConnessione")`	Recupera la stringa di connessione dal file di configurazione (`App.config` / `Web.config`) e restituisce un oggetto `SqlConnection`. Lancia un'eccezione se la stringa non esiste.
Scalar	`ExecuteScalar(query As String, Optional parameters As List(Of SqlParameter) = Nothing,`	Esegue una query che restituisce un singolo valore (es. `COUNT`, `MAX`). Restituisce l'oggetto risultante.

Sezione	Funzione	Descrizione
	`Optional connectionName As String = "MiaConnessione")`	
Non-Query	`ExecuteNonQuery(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Esegue `INSERT`, `UPDATE` o `DELETE`. Restituisce il numero di righe interessate. Se il risultato è `0`, scrive un avviso su `Debug`.
Reader	`ExecuteReader(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Restituisce un `SqlDataReader` con `CommandBehavior.CloseConnection` per chiudere automaticamente la connessione al termine della lettura.
DataTable	`FillDataTable(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Riempie e restituisce un `DataTable` (utile per visualizzazioni in UI come DataGridView).
DataSet	`FillDataSet(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")`	Simile a `FillDataTable` ma per più tabelle (riempie un `DataSet`).
Utility Parametri	`AddParameters(cmd As SqlCommand, parameters As List(Of SqlParameter))`	Aggiunge un array di parametri a un `SqlCommand`.
Query semplificata	`Query(sql As String, ParamArray values() As Object)`	Crea automaticamente parametri `@p1`, `@p2`, … a partire dagli argomenti passati e restituisce un `DataTable`.
Query LIKE	`QueryLike(sql As String, ParamArray values() As String)`	Avvolge `Query` aggiungendo i caratteri `%` ai valori per le clausole `LIKE`.
Metadati	`GetTables()`	Restituisce un `DataTable` con i nomi delle tabelle di base presenti nel database.

---

1. **Chiamata di un metodo** (es. `ExecuteNonQuery`).

2. **Connessione**
   - `GetConnection` recupera la stringa di connessione dal file di configurazione.
   - Viene creato un oggetto `SqlConnection` e aperto (`conn.Open()`).
3. **Comando**
   - Viene istanziato un `SqlCommand` con la query e la connessione.
   - Se sono presenti parametri, `AddParameters` li aggiunge al comando.
4. **Esecuzione**
   - `ExecuteNonQuery`, `ExecuteScalar`, `ExecuteReader`, `SqlDataAdapter.Fill`, ecc. vengono chiamati in base al metodo richiesto.
5. **Pulizia**
   - I blocchi `Using` garantiscono la chiusura e il rilascio delle risorse (`conn`, `cmd`, `da`).
   - Per `ExecuteReader`, la chiusura avviene automaticamente grazie a `CommandBehavior.CloseConnection`.

---

4. Punti di forza

> **Punti di forza**
>
> - **Semplicità**: i metodi richiedono solo la query e, opzionalmente, i parametri.
> - **Sicurezza**: l'uso di `SqlParameter` previene l'injection SQL.
> - **Riutilizzabilità**: tutti i metodi sono statici, quindi non è necessario creare istanze.
> - **Gestione automatica delle risorse**: i blocchi `Using` evitano perdite di connessioni.

---

5. Esempio di utilizzo

```
' Inserimento di un nuovo cliente
Dim sqlInsert = "INSERT INTO Clienti (Nome, Citta) VALUES (@p1, @p2)"
Dim rows = DB.ExecuteNonQuery(sqlInsert, New List(Of SqlParameter) From
{
    New SqlParameter("@p1", "Mario"),
    New SqlParameter("@p2", "Roma")
})

' Recupero di tutti i clienti
Dim dt = DB.FillDataTable("SELECT * FROM Clienti")

' Ricerca con LIKE
```

```
12  Dim dtLike = DB.QueryLike("SELECT * FROM Clienti WHERE Citta LIKE @p1",
    "Roma")
```

> **ⓘ Conclusione** 📘
>
> La classe `DB` è un helper di accesso ai dati che semplifica notevolmente la scrittura di codice di persistenza in VB.NET. Fornisce un’interfaccia coerente e sicura per eseguire query, gestire parametri e ottenere risultati in vari formati, riducendo il rischio di errori e migliorando la manutenibilità del progetto.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del Codice VB.NET

1. Scopo Generale

Il file **DBQry.vb** contiene due classi (`DBApri` e `DataGVLoad`) che forniscono un'interfaccia semplice e centralizzata per:

1. **Gestire le connessioni a un database SQL Server** tramite una stringa di connessione definita nel file di configurazione (`app.config` o `web.config`).
2. **Eseguire query SQL** (SELECT, INSERT, UPDATE, DELETE) e restituire i risultati in forma di `DataTable` , valore scalare o numero di righe interessate.
3. **Popolare un controllo** `DataGridView` con i dati ottenuti da una query.

In pratica, il codice è un "wrapper" di accesso ai dati che riduce la duplicazione di codice e centralizza la gestione degli errori.

---

2. Componenti Chiave

Classe	Funzione	Descrizione
DBApri	`GetConnection()`	Restituisce una nuova istanza di `SqlConnection` usando la stringa di connessione `MiaConnessione` dal file di configurazione.
	`GetDataTable(query, parameters)`	Esegue una query SELECT e restituisce i risultati in un `DataTable` . Accetta parametri opzionali.
	`ExecuteScalar(query, parameters)`	Esegue una query che restituisce un singolo valore (es. `COUNT(*)`).

Classe	Funzione	Descrizione
	`ExecuteNonQuery(query, parameters)`	Esegue una query che non restituisce risultati (INSERT, UPDATE, DELETE) e ritorna il numero di righe interessate.
DataGVLoad	`ApriDGV(dgv, query, parameters)`	Popola un `DataGridView` con i risultati di una query. Gestisce eccezioni mostrando un `MessageBox`.

Dettagli Implementativi

- `Imports`:
  - `System.Configuration` → per leggere le stringhe di connessione.
  - `System.Data.SqlClient` → per l'API ADO.NET di SQL Server.
  - `System.Windows.Forms` → per l'interfaccia grafica (DataGridView, MessageBox).
- `NotInheritable`: la classe `DBApri` è dichiarata `NotInheritable` (sealed) per evitare ereditarietà indesiderata.
- `Private Sub New()`: costruttore privato che impedisce l'istanza della classe, poiché tutti i metodi sono `Shared`.
- `Using`: blocchi `Using` garantiscono la chiusura e la liberazione delle risorse (`SqlConnection`, `SqlCommand`, `SqlDataAdapter`) anche in caso di eccezioni.
- `Optional parameters As List(Of SqlParameter) = Nothing`: permette di passare parametri SQL in modo sicuro (prevenendo SQL injection) o di ometterli se non necessari.
- `LogLeggiScrivi.ScriviLog`: chiamata all'interno di `GetDataTable` per registrare eventuali errori. (Non è definita nel file, ma presumibilmente è una classe di logging personalizzata.)
- `MessageBox.Show`: visualizza un messaggio di errore all'utente quando `ApriDGV` fallisce.

---

3. Flusso di Esecuzione

3.1. Ottenere una Connessione

```
Using conn = GetConnection()
    conn.Open()
    ...
End Using
```

- `GetConnection()` legge la stringa di connessione dal file di configurazione e crea un nuovo `SqlConnection`.
- `conn.Open()` apre la connessione al database.

3.2. Eseguire una Query

a) `GetDataTable`

1. **Creazione del comando**: `New SqlCommand(query, conn)`.
2. **Aggiunta di parametri** (se presenti): `cmd.Parameters.AddRange(parameters.ToArray())`.
3. **DataAdapter**: `New SqlDataAdapter(cmd)` per riempire un `DataTable`.
4. **Riempimento**: `da.Fill(dt)`.
5. **Restituzione**: il `DataTable` viene restituito al chiamante.

b) `ExecuteScalar`

1. Simile a `GetDataTable`, ma invece di riempire un `DataTable`, si chiama `cmd.ExecuteScalar()` che restituisce il primo valore della prima riga del risultato.

c) `ExecuteNonQuery`

1. Simile a `ExecuteScalar`, ma chiama `cmd.ExecuteNonQuery()` che restituisce il numero di righe colpite dalla query.

3.3. Popolare un DataGridView

```
1  DataGVLoad.ApriDGV(dgvClienti, "SELECT * FROM Clienti")
```

> **Info**
>
> 1. Verifica che `dgv` non sia `Nothing`.
> 2. Chiama `DBApri.GetDataTable(query, parameters)` per ottenere i dati.
> 3. Imposta `dgv.DataSource` al `DataTable` restituito.
> 4. In caso di eccezione, mostra un `MessageBox` con l'errore.

---

4 Flusso di esecuzione

1. **Richiesta di dati**
   L'applicazione chiama `DataGVLoad.ApriDGV(dgv, query, parameters)`.
2. **Validazione**
   Se `dgv` è `Nothing`, viene lanciata un'eccezione.
3. **Esecuzione della query**
   `ApriDGV` chiama `DBApri.GetDataTable(query, parameters)`.
4. **Connessione al database**
   - `GetConnection` crea una `SqlConnection` con la stringa di connessione.

- `conn.Open()` apre la connessione.

5. **Comando SQL**
   - Viene creato un `SqlCommand` con la query e la connessione.
   - Se presenti, i parametri vengono aggiunti al comando.
6. **Recupero dati**
   - Un `SqlDataAdapter` esegue il comando e riempie un `DataTable`.
   - Il `DataTable` viene restituito a `ApriDGV`.
7. **Aggiornamento UI**
   - `dgv.DataSource` viene impostato sul `DataTable`, facendo apparire i risultati nella griglia.
8. **Gestione eccezioni**
   - Se `GetDataTable` lancia un'eccezione, `ApriDGV` la cattura e mostra un messaggio di errore all'utente.

Il codice fornisce un set di metodi statici per:

- **Connettersi** a SQL Server in modo centralizzato.
- **Eseguire** query di lettura, scrittura e recupero di singoli valori.
- **Popolare** controlli UI (`DataGridView`) con i risultati.

> **Conclusioni**
>
> La struttura è pulita, utilizza `Using` per la gestione delle risorse e supporta parametri SQL per sicurezza. Effettuato ottimizzazione (logging, gestione degli errori, separazione UI/logic) il modulo diventa ancora più robusto e riutilizzabile.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del Codice `GridFilter`

1. Scopo Generale

Il modulo `GridFilter` fornisce due metodi statici (`Shared`) che permettono di filtrare e di ripristinare il filtro di una `DataGridView` in un'applicazione Windows Forms.
Il filtro è applicato in modo dinamico al *data source* della griglia, indipendentemente dal tipo di oggetto di origine (BindingSource, DataView o DataTable).

---

2. Componenti Chiave

Metodo	Descrizione	Parametri	Operazioni principali
`FiltraDgv`	Applica un filtro alla griglia.	`dgv As DataGridView` , `filtro As String`	1. Determina il tipo di `DataSource` . 2. Imposta la proprietà di filtro corrispondente.
`ResetFiltro`	Rimuove qualsiasi filtro impostato.	`dgv As DataGridView`	1. Determina il tipo di `DataSource` . 2. Resetta la proprietà di filtro a stringa vuota.

2.1 Tipi di DataSource gestiti

1. `BindingSource` – gestisce la visualizzazione di una collezione di oggetti.
2. `DataView` – vista filtrata di una `DataTable` .
3. `DataTable` – tabella di dati in memoria; il filtro viene applicato tramite `DefaultView.RowFilter` .

---

3. Flusso di Esecuzione

3.1 Metodo `FiltraDgv`

1. **Tentativo di cast a `BindingSource`**

```
Dim bs = TryCast(dgv.DataSource, BindingSource)
If bs IsNot Nothing Then
    bs.Filter = filtro
    Exit Sub
End If
```

   - Se il `DataSource` è un `BindingSource`, la proprietà `Filter` viene impostata con la stringa `filtro` e il metodo termina.

2. **Tentativo di cast a `DataView`**

```
Dim dv = TryCast(dgv.DataSource, DataView)
If dv IsNot Nothing Then
    dv.RowFilter = filtro
    Exit Sub
End If
```

   - Se il `DataSource` è un `DataView`, la proprietà `RowFilter` viene impostata.

3. **Tentativo di cast a `DataTable`**

```
Dim dt = TryCast(dgv.DataSource, DataTable)
If dt IsNot Nothing Then
    dt.DefaultView.RowFilter = filtro
    Exit Sub
End If
```

   - Se il `DataSource` è una `DataTable`, il filtro viene applicato alla sua `DefaultView`.

4. **Fine** – Se nessun cast ha avuto successo, il metodo termina senza modificare nulla.

3.2 Metodo `ResetFiltro`

Il flusso è identico a `FiltraDgv`, ma la proprietà di filtro viene impostata a una stringa vuota (`""`) per rimuovere qualsiasi filtro precedente.

---

4 . Conclusione

Il codice `GridFilter` è una soluzione semplice e riutilizzabile per filtrare e ripristinare i dati visualizzati in una `DataGridView`.
Gestisce i tre casi più comuni di origine dati e offre un'interfaccia uniforme, consentendo agli sviluppatori di applicare filtri dinamicamente senza preoccuparsi del tipo sottostante di `DataSource`.

Con alcune piccole ottimizzazioni (gestione di errori, supporto per altri tipi di origine dati), il modulo può diventare ancora più robusto e versatile.

Esempio utilizzo

Scopo generale del codice

Il frammento è parte di un form Windows Forms scritto in **VB.NET**.
Si occupa di gestire due eventi dell'interfaccia utente:

1. **Cerca** – evidenzia in una griglia (`FatturaDataGrid`) le celle che contengono il testo digitato dall'utente.
2. **Filtra** – filtra dinamicamente la stessa griglia in base al contenuto di un textbox di ricerca globale (`TxtTutti`).

Componenti principali

Elemento	Descrizione
`RjBtnCerca`	Bottone che avvia la ricerca.
`TxtEvidenzia`	TextBox dove l'utente inserisce il termine da evidenziare.
`TxtTutti`	TextBox per filtrare l'intera griglia in tempo reale.
`FatturaDataGrid`	DataGridView che mostra i dati delle fatture.
`dtOriginal`	DataTable originale (non filtrata) usata come base per il filtro.
`GridUtility`	Classe di utilità (non mostrata) che contiene i metodi `EvidenziaTesto` e `FiltraTutti` .

Funzioni chiave

- `GridUtility.EvidenziaTesto(grid, filtro)`
  Evidenzia tutte le celle del `grid` che contengono la stringa `filtro` .
  Presumibilmente cambia il colore di sfondo o il font delle celle corrispondenti.
- `GridUtility.FiltraTutti(grid, dataTable, filtro)`
  Applica un filtro alla `dataTable` e aggiorna il `grid` per mostrare solo le righe che soddisfano il criterio.
  Il filtro è probabilmente una ricerca "contiene" su tutti i campi.

Flusso di esecuzione

1. Evento `RjBtnCerca_Click`

1. **Lettura del testo**

```
1  Dim filtro As String = TxtEvidenzia.Text.Trim.ToUpper()
```

Si ottiene il contenuto del textbox, si rimuovono spazi iniziali/finali e si converte in maiuscolo per una ricerca case-insensitive.

2. **Validazione**
   Se `filtro` è vuoto, viene mostrato un messaggio di avviso e la sub termina con `Return`.
3. **Evidenziazione**
   Se il filtro è valido, si chiama

```
GridUtility.EvidenziaTesto(FatturaDataGrid, filtro)
```

   che evidenzia le celle corrispondenti.

2. Evento `TxtTutti_TextChanged`

Ogni volta che l'utente modifica il testo in `TxtTutti`:

```
GridUtility.FiltraTutti(FatturaDataGrid, dtOriginal, TxtTutti.Text)
```

Il metodo filtra la griglia in base al nuovo valore, aggiornando la visualizzazione in tempo reale.

> **Conclusione**
>
> Il codice fornisce una semplice interfaccia di ricerca e filtraggio per una griglia di fatture, sfruttando metodi di utilità per separare la logica di visualizzazione dalla logica di business. È chiaro, leggibile e facilmente estendibile.



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET V2.0.6 DOWNLOADS 1.3K LICENSE MIT GITHUB PAGES ONLINE

Analisi del codice `GridUtility`

1. Scopo generale

Il file `GridUtility.vb` contiene una classe **utility** (`GridUtility`) dedicata alla configurazione e manipolazione di oggetti `DataGridView` in un'applicazione Windows Forms.
La classe è **non ereditabile** (`NotInheritable`) e fornisce due metodi pubblici statici:

1. `Initialize(dgv As DataGridView)` – imposta le proprietà di stile e comportamento di un `DataGridView` in modo coerente con il tema dell'applicazione.
2. `ConvertiMaiuscolo(dgv As DataGridView)` – converte tutti i valori di tipo stringa presenti nelle celle in maiuscolo.

Questa struttura permette di centralizzare la logica di formattazione, riducendo la duplicazione di codice in più form o controlli.

---

2. Componenti chiave e funzioni

Elemento	Descrizione	Note
Costruttore privato	`Private Sub New()`	Previene l'istanza della classe; tutti i metodi sono statici.
Font statici	`FontBold` , `FontRegular`	Definiscono due font predefiniti (bold e regular) da usare nei DataGridView.
`Initialize`	Configura le proprietà di base, l'aspetto dell'header, le righe, le righe alternate, l'altezza delle righe e i colori di selezione.	Utilizza classi di supporto (`Colori` , `ThemeFonts` , `FrameworkLogger`) che non sono incluse nel snippet.

Elemento	Descrizione	Note
`ConvertiMaiuscolo`	Scorre tutte le righe e le celle, trasformando i valori stringa in maiuscolo.	Gestisce eccezioni e logga eventuali errori.
`FrameworkLogger`	Classe di logging (presumibilmente).	Non è definita qui, ma è usata per registrare errori.
`Colori`	Classe di utilità per ottenere colori definiti dal tema.	Metodo `GetColor` restituisce un `Color`.
`ThemeFonts`	Classe di utilità per ottenere font predefiniti.	Metodo `DefaultFontBold` restituisce un `Font`.

---

3. Flusso di esecuzione

3.1 `Initialize`

1. **Controllo di nullità**

```
1  If dgv Is Nothing Then Exit Sub
```

   Se il parametro è `Nothing`, la routine termina subito.
2. **Configurazione di base**
   - Disabilita lo stile visuale degli header.
   - Imposta il bordo a `None`.
   - Disabilita l'aggiunta/eliminazione di righe da parte dell'utente.
   - Imposta la selezione a singola riga (`FullRowSelect`).
   - Imposta l'auto-ridimensionamento delle colonne a `Fill`.
   - Nasconde le intestazioni di riga e rende il controllo **read-only**.
   - Imposta colori di sfondo e griglia tramite `Colori.GetColor`.
3. **Stile dell'header**
   - Colore di sfondo e testo.
   - Font in grassetto (`ThemeFonts.DefaultFontBold`).
   - Allineamento centrato.
   - Altezza header a 35 pixel.
4. **Stile delle righe**
   - Font di default (`ThemeFonts.DefaultFont`).
   - Colori di testo, sfondo e selezione.
   - Colore di riga alternata a `WhiteSmoke`.
   - Altezza riga a 30 pixel.
5. **Rimozione del colore di selezione "blu Windows"**

- Imposta nuovamente `SelectionBackColor` e `SelectionForeColor` per sovrascrivere il colore di default di Windows.

6. **Gestione eccezioni**
   - Tutto il blocco è avvolto in `Try/Catch`.
   - In caso di errore, viene chiamato `FrameworkLogger.LogError`.

3.2 `ConvertiMaiuscolo`

1. **Controlli preliminari**
   - Se il `DataGridView` è `Nothing` o non contiene righe, la routine termina.
2. **Iterazione su righe e celle**

```
For Each row As DataGridViewRow In dgv.Rows
    For Each cell As DataGridViewCell In row.Cells
        If TypeOf cell.Value Is String Then
            cell.Value = cell.Value.ToString().ToUpper()
        End If
    Next
Next
```

   - Per ogni cella, se il valore è una stringa, viene convertito in maiuscolo con `ToUpper()`.
3. **Gestione eccezioni**
   - Come `Initialize`, l'intero ciclo è protetto da `Try/Catch` e logga eventuali errori.

---

4. Come usarlo

```
' Inizializza un DataGridView con lo stile predefinito
GridUtility.Initialize(Me.ClientiDataGridView)

' Converte tutti i valori stringa in maiuscolo
GridUtility.ConvertiMaiuscolo(Me.ClientiDataGridView)
```

Questa utility rende più semplice mantenere un aspetto coerente tra tutti i DataGridView dell'applicazione e fornisce una funzione di utilità comune (convertire in maiuscolo).

Spiegazione del Codice Evidenzia Testo

1. Scopo Generale

Il metodo `EvidenziaTesto` è una procedura condivisa (static) che permette di evidenziare in un `DataGridView` tutte le celle che contengono una determinata stringa di testo. L'evidenziazione avviene cambiando il colore di sfondo delle celle che soddisfano il criterio di ricerca.

2. Componenti Chiave

Elemento	Descrizione	Note
`dgv As DataGridView`	Il controllo `DataGridView` su cui operare.	Deve essere già inizializzato e popolato.
`testo As String`	Testo da cercare all'interno delle celle.	La ricerca è case-insensitive.
`Optional colore As Integer = 5`	Indice di colore (default 5).	Viene passato a `Colori.GetColor` per ottenere il colore effettivo.
`Try…Catch`	Blocco di gestione eccezioni.	In caso di errore, viene registrato tramite `FrameworkLogger.LogError`.
`Colori.GetColor(colore)`	Funzione (presumibilmente) che restituisce un oggetto `Color` a partire dall'indice.	Non è mostrata nel frammento, ma è responsabile della conversione.
`FrameworkLogger.LogError`	Logger personalizzato.	Serve per tracciare errori senza interrompere l'applicazione.

3. Flusso di Esecuzione

1. **Validazione di input**
   - Se `dgv` è `Nothing` o non contiene righe (`Rows.Count = 0`), la procedura termina subito.
   - Se `testo` è `Nothing`, vuoto o contiene solo spazi, la procedura termina.
2. **Iterazione sulle righe e sulle celle**
   - Per ogni riga (`DataGridViewRow`) del `DataGridView`.
   - Per ogni cella (`DataGridViewCell`) della riga corrente.
3. **Controllo del valore della cella**
   - Se la cella ha un valore (`cell.Value IsNot Nothing`) e quel valore, convertito in stringa, contiene la stringa `testo` (ricerca case-insensitive grazie a `StringComparison.OrdinalIgnoreCase`), allora:
     - Viene impostato il colore di sfondo della cella con `cell.Style.BackColor = Colori.GetColor(colore)`.
4. **Gestione eccezioni**

- Qualsiasi eccezione sollevata durante l'esecuzione viene catturata e loggata con `FrameworkLogger.LogError`.

---

4. Esempio di Utilizzo

```
' Evidenzia tutte le celle che contengono la stringa "RAM" nel DataGridView DgvOrdini
GridUtility.EvidenziaTesto(DgvOrdini, "RAM")
```

In questo esempio, tutte le celle che contengono "RAM" (in qualsiasi combinazione di maiuscole/minuscole) verranno colorate con il colore associato all'indice 5.

Conclusione

Il metodo `EvidenziaTesto` è una utility semplice e utile per evidenziare rapidamente contenuti specifici in un `DataGridView`. Con alcune ottimizzazioni e miglioramenti di robustezza, può diventare una componente affidabile di un'applicazione Windows Forms.

Analisi del codice Filtra Tutti

1. Scopo generale

Il file contiene una classe di utilità (`GridUtility` – non mostrata ma implicita) che fornisce metodi statici per:

1. **Filtrare** i dati visualizzati in un `DataGridView` in base a un testo di ricerca.
2. **Colorare** lo sfondo di celle che contengono valori null o vuoti.
3. **Estrarre** il `DataTable` originale da un `DataGridView` indipendentemente dal tipo di `DataSource` (DataTable, DataView, BindingSource).

Queste funzioni sono pensate per semplificare le operazioni comuni di interfaccia utente in applicazioni WinForms che mostrano dati tabellari.

---

2. Componenti principali

Metodo	Parametri	Funzione
`FiltraTutti`	`dgv As DataGridView`, `dtOriginal As DataTable`, `testo As String`	Filtra tutti i campi di `dtOriginal` con la stringa `testo` e imposta il risultato come `DataSource` del `DataGridView`.

Metodo	Parametri	Funzione
`FiltraTesto`	`dgv As DataGridView`, `testo As String`	Recupera il `DataTable` associato al `DataGridView` (indipendentemente dal tipo di `DataSource`) e chiama `FiltraTutti`.
`ImpostaColNull`	`dgv As DataGridView`, `nomeColonna As String`, `backColor As Color = Nothing`, `foreColor As Color = Nothing`	Evidenzia le celle della colonna indicata che contengono valori null o vuoti, usando colori di default (giallo/azzurro) se non specificati.

Dettagli di implementazione

- **Sicurezza**: tutti i metodi verificano che l'oggetto `dgv` non sia `Nothing` e che le colonne esistano prima di procedere.
- **Gestione degli errori**: `FiltraTutti` è avvolto in un blocco `Try/Catch` che registra l'eccezione tramite `FrameworkLogger.LogError`.
- **Costruzione del filtro**: per ogni colonna di `dtOriginal` si crea una clausola `LIKE` (per stringhe) o una conversione a stringa (per altri tipi) e si concatenano con `OR`.

```
1  filtro &= $"[{col.ColumnName}] LIKE '%{testo.Replace("'", "''")}%'"
```

- **DataView**: viene creato un nuovo `DataView` con `RowFilter` impostato al filtro costruito, e questo `DataView` diventa la nuova fonte dati del `DataGridView`.
- **Colorazione**: il metodo scorre tutte le righe (escluse le righe di inserimento) e controlla se il valore è `Nothing`, `DBNull` o una stringa vuota. Se sì, imposta i colori di sfondo e testo; altrimenti ripristina i colori di default del `DataGridView`.

---

3. Flusso di esecuzione

3.1 `FiltraTutti`

1. **Validazione**: se `dgv` o `dtOriginal` sono `Nothing`, il metodo termina subito.
2. **Ricerca vuota**: se `testo` è vuoto o solo spazi, il `DataSource` viene riportato a `dtOriginal` (tutti i record) e il metodo termina.
3. **Costruzione del filtro**:
   - Inizializza `filtro` come stringa vuota.
   - Per ogni colonna di `dtOriginal`:
     - Se non è la prima colonna, aggiunge `" OR "`.
     - Se la colonna è di tipo `String`, aggiunge una clausola `LIKE` con escape per l'apostrofo.
     - Altrimenti, converte la colonna a stringa e applica `LIKE`.

4. **Applicazione del filtro**: crea un `DataView` su `dtOriginal`, imposta `RowFilter` al filtro costruito e assegna il `DataView` al `DataSource` del `DataGridView`.
5. **Gestione eccezioni**: eventuali errori vengono registrati.

3.2 `FiltraTesto`

1. **Recupero del DataTable**: controlla il tipo di `DataSource` del `DataGridView` (DataTable, DataView, BindingSource) e ottiene il `DataTable` corrispondente.
2. **Chiamata di `FiltraTutti`**: se il `DataTable` è stato trovato, chiama `FiltraTutti(dgv, dt, testo)` (commentato come "Versione veloce").
3. **Fine**: se non è stato trovato un `DataTable`, il metodo termina senza fare nulla.

3.3 `ImpostaColNull`

1. **Validazione**: verifica che `dgv` non sia `Nothing` e che la colonna esista.
2. **Colori di default**: se `backColor` o `foreColor` non sono specificati, li impostano rispettivamente a `Yellow` e `Blue`.
3. **Iterazione delle righe**: per ogni riga non nuova:
   - Recupera il valore della cella nella colonna indicata.
   - Determina se è null/DBNULL/empty.
   - Se è null, imposta i colori di sfondo e testo; altrimenti ripristina i colori di default del `DataGridView`.

---

4. Conclusione

Il codice fornisce un set di strumenti utili per la gestione di `DataGridView` in WinForms:

> **Note**
>
> - **Filtraggio dinamico** di tutti i campi con una singola stringa di ricerca.
> - **Estrazione** del `DataTable` originale da varie tipologie di `DataSource`.
> - **Evidenziazione** visiva di celle con valori null o vuoti.
> - Aggiunta XML in tutte le funzioni
>
> Documentazione più completa, questi metodi sono parte integrante di una libreria di utilità UI.